

Fonctionnement du benchmark

Objectif

BenchGo V3 evalue un modele LLM local sur des taches de code reelles.

Il ne se contente pas de verifier des mots cles: il execute aussi le code quand c est possible.

Pipeline d execution

1. Lecture des arguments CLI
2. Choix du profil (force ou auto-detecte)
3. Chargement des tiers JSON
4. Envoi du prompt au modele via LM Studio
5. Parsing de la reponse JSON
6. Evaluation des taches (exec, pattern, custom)
7. Calcul des scores
8. Generation du rapport Markdown
9. Ecriture des logs persistants

Types d evaluation

- exec:
 - execute du code en VM sandbox
 - compare resultat obtenu vs attendu
- pattern:
 - verifie motifs requis/interdits dans le code
- custom:
 - evaluateurs specialises pour cas complexes

Profils d evaluation

- LIGHT:
 - tiers obligatoires: 0, 1
 - tiers optionnels: 2, 3
- STANDARD:
 - tiers obligatoires: 0, 1, 2
 - tiers optionnel: 3

- EXPERT:
 - tiers obligatoires: 0, 1, 2, 3
 - aucun tier optionnel

Detection automatique du profil

Si vous ne passez pas `--profile`, BenchGo tente:

1. GET `http://localhost:1234/v1/models`
2. Extraction de la taille du modele dans son nom
3. Mapping:
 - `< 3B => LIGHT`
 - `3B a 14B => STANDARD`
 - `> 14B => EXPERT`

Si echec, STANDARD est applique.

Gestion de la fenetre de contexte

Le client calcule un budget de sortie `max_tokens` en fonction de:

- prompt systeme
- prompt utilisateur
- `--context-limit`

Si le prompt est trop proche de la limite, la requete est arretee avec erreur explicite.

Rattrapage

Disponible pour LIGHT et STANDARD:

- en cas d echec de tier
- seulement en terminal interactif
- une seule tentative supplementaire maximum
- conservation du meilleur score

Fichiers d entree et de sortie

Entrees:

- `benchmark-v2/tiers/*.json`

Sorties:

- rapport final a la racine du projet (rapportv3...md)
- logs dans benchmark-v2/logs/benchmarks-v2-YYYYMMDD-HHmmss.log